# The secret scan (GitGuardian): what it is, and how to help when someone forgets

*One check stops a commit. Everything else advises. This is that check, and what to do when it speaks.*

Kind Walkthrough | Changed 2026-11-04 | For Anyone who commits and their AI helper

**You are here:** drasticstatic lab › `my-template` › `walkthroughs` › `SECRET-SCAN.md` · public · `main` is the working branch

## TL;DR

1. **GitGuardian's `ggshield` checks every change for secrets** (passwords, API keys, private keys, tokens).
2. **It runs in two places, and it's the one check that stops a commit.** Everything else advises.

| Where | Who sets it up | What it needs | If it's missing |
|---|---|---|---|
| **On your computer**, before a commit leaves it (the `pre-commit` hook) | **Each person, once, optional** | Your own free GitGuardian login: `ggshield auth login` | The hook prints a warning and the commit goes ahead. The CI scan still checks it |
| **On GitHub**, on every push and pull request (the Guardrails workflow) | **An owner, once per private workshop repository** | One repository secret, `GITGUARDIAN_API_KEY`, from the organization's own GitGuardian account | The workflow warns "Secret scan skipped" and nothing is scanned in CI. **Fix this first** |

3. **Nobody needs a key of their own to contribute.** One key per workshop repository covers every person and every AI helper. Public repositories only receive the finished website, so they don't need one.

## For the AI helper: how to help your person

You'll usually meet the secret scan in one of these moments. Match the message, then follow the row.

| What you see | What it means | What to do |
|---|---|---|
| `⚠ ggshield not found on PATH — secret scan skipped for this commit.` | ggshield isn't installed on this computer | The commit went through, so nothing is wrong yet. Offer to set it up (below). The CI scan is still the safety net |
| `ggshield` asks for authentication / *no API key* | Installed, but this person hasn't logged in | Ask your person to run `ggshield auth login` themselves (it opens their browser). **Never ask them to paste a key into chat** |
| `ggshield` reports **secrets found** and the commit stops | A secret is in the staged change | **Stop. Do not bypass.** Follow *"A secret was found"* below |
| A Guardrails run says `Secret scan skipped: add the GITGUARDIAN_API_KEY secret` | The repository has no CI key yet | Tell your person that **an owner** adds it (below). Until then, only local scans protect this repository |
| It still says **skipped**, though the secret shows in the repository's list | The secret was saved **empty** (see the gotcha below) | Have your person set it again on the repository's Secrets page, then re-run Guardrails |

| | | |
|---|---|---|
| The Guardrails **Secret scan** step fails after a push | A secret reached GitHub | Treat it as leaked: follow *"A secret reached GitHub"* below. Tell your person straight away |
| No hooks ran at all on a fresh clone | Hooks aren't activated in this clone | Run `sh scripts/install-hooks.sh` (it sets `core.hooksPath` to `.githooks/`) |
| A change was made on GitHub's website, by Codex in the cloud, or by ChatGPT | Local hooks never ran | That's expected. After it lands, check that the Guardrails run is green |

**Never, under any circumstance:**

- `git commit --no-verify` (or `-n`).
- Unset `core.hooksPath`, or delete or empty `.githooks/pre-commit`.
- Read, print or echo the secret itself back into the conversation. Name the **file and line** instead.
- Mark something as a false positive without your person's clear yes.

## Set up the local scan (each person, once, optional)

Have your person run these themselves, in their own terminal:

```
brew install ggshield        # macOS with Homebrew
# or: pipx install ggshield  # anywhere with Python

ggshield auth login          # opens the browser; log in or create a free account
```

`ggshield auth login` creates a personal access token and stores it in ggshield's own configuration. **Nobody copies or pastes it anywhere.** To check that it worked: `ggshield api-status`.

Then, in each repository clone, activate the fleet's hooks once:

```
sh scripts/install-hooks.sh
```

The repository's `.githooks/pre-commit` calls `ggshield secret scan pre-commit` on every commit. It also re-chains any older local hook. This matters because the fleet's own hooks path would otherwise switch off ggshield's global hook without a word.

## Set up the CI scan (an owner, once per workshop repository)

1. Sign in to the **organization's** GitGuardian account (owned by the organization's email, not a person's).
2. Create a **personal access token** with the **scan** scope.
3. In the private workshop repository: **Settings** → **Secrets and variables** → **Actions** → **New repository secret.** Name it `GITGUARDIAN_API_KEY` and paste the token **there**, never into a chat or a file.
4. Push any change and check that the Guardrails run shows the **Secret scan** step running, not skipped.

> ***Gotcha: a secret can be saved empty without a word.*** *`gh secret set NAME -R <owner>/<repo>` asks for the value at a hidden prompt. In a shell that **can't** prompt, it stores an **empty** secret and reports success. That includes an AI helper's shell, Claude Code's `!` shell and scripts. The name appears in the list, but the scan still says skipped. Use one of these instead:*
>
> - *the repository's **Secrets** page in the browser, or*
> - *your own Terminal, or*

> - *a pipe from the clipboard, which works anywhere:* `pbpaste | gh secret set NAME -R <owner>/<repo> && pbcopy </dev/null` .
>
> *Then re-run Guardrails and look for the scan's own result line.*

One key per workshop repository. The public `-publish-preview` and `-public` repositories don't need one.

## A secret was found (the commit stopped)

The secret has **not** left the computer. Good: the scan did its job.

1. **Find it.** ggshield names the file and line. Tell your person which file, never the value.
2. **Take it out of the change:** `git restore --staged <file>` , then remove the secret from the file.
3. **Put it where secrets belong:**
   - For local use: a git-ignored `.env.local` file.
   - For a workflow: a GitHub Actions secret, which an owner adds in the repository's settings.
   - Code reads it from the environment ( `process.env.NAME` ), never from a written-in value.
4. **Commit again.** The scan runs again and should pass.

**Is it a false positive?** Some values look like secrets but aren't, such as a public client ID or a published test number. That's **your person's** call, not yours. If they say yes:

```
ggshield secret ignore --last-found
```

This records the ignored match in `.gitguardian.yaml` . Commit that file **in its own commit**, with a message saying what was ignored and why, so reviewers can see it.

## A secret reached GitHub (the CI scan failed)

Once a secret has been pushed, **assume someone else has seen it**, even in a private repository.

1. **Rotate it first.** In the service it belongs to, revoke that key or password and create a new one. Your person does this; it's the step that actually makes them safe.
2. **Then remove it** from the code in a new commit, and put the new value where secrets belong (above).
3. **Rewriting history is an owner's decision,** because removing a value from older commits needs a force push. Rotation already made the old value useless, so a rewrite is rarely needed.
4. **Write a short note** in the repository's `AGENT-SYNC/` saying what leaked, when, that it's rotated, and where the new value lives. Never write the value itself.

## Questions people ask

**▼ Does everyone who commits need a GitGuardian account?**

No. The CI key covers every person and every helper. A personal login only adds the earlier, on-computer check, which is nice to have.

**▼ Why does the scan stop commits when the credit line never does?**

A missing credit line can be fixed with a note. A leaked key can be used by strangers within minutes. That difference is why the secret scan is the one hard stop.

**▼ Codex / ChatGPT / GitHub's website made the change. Was it scanned?**

Not on a computer, because those tools don't run local hooks. The CI scan checks it on GitHub. That's exactly why the CI key matters most.

▼ **Is a wallet address a secret?**

A public address isn't a secret, but **our rule is still: no real or made-up wallet addresses** until the organization provides its own. A seed phrase or private key **is** a secret, always. Never write one anywhere.

## Suggest a change

Noticed something? Open an issue or a pull request against `main`, changing this page; a steward reads it and a person with the role merges, with a merge commit, so your name stays on it. An AI helper may do all of that up to the merge, signed as the attribution spec says. Not on GitHub? Tell any steward you know, or your helper, in your own words.

Part of the walkthroughs. Official docs: ggshield getting started. Icons: Lucide (ISC).